```
echo '\'
```

both the above will produce the following output:

Moving on to Double quotes, they will suppress special meaning of almost every meta characters except $, \ and `. Rest of all the meta characters will be read literally. It is also possible to use single quote within double quotes.

```
#!/bin/bash
#use of single quotes within double quotes
echo "see I won't put single quotes here"
```

```
subiet@999-Workstation:/media/Data/Projects/Digit/Linux Admin$ ./test.sh
see I won't put single quotes here
```

Without single quotes

will produce the following:

Functions and More

A function is declared by using the `function` keyword in shell scripting followed by the function name.

```
#!/bin/bash
# Declaring a function
function digit_func
{
     echo 'Hey! We are inside a function.'
}
digit_func
```

Please note the brackets after the fuction name, which in most programming languages enclose the parameter list. This is because the

```
subiet@999-Workstation:/media/Data/Projects/Digit/Linux Admin$ ./test.sh
Hey! We are inside a function.
subiet@999-Workstation:/media/Data/Projects/Digit/Linux Admin$
```

Your first function

paramaters are by default assigned variable names as $1, $2 and so on while $0 will refer to the script name by default. This greatly increases the speed of programming at the cost of slight loss of flexibility.

Parameters are passed to a function by merely separating them with a space next to the function call. To illustrate what we are saying, consider this:

```
#!/bin/bash
# Declaring a function
function digit_func
{
     echo 'Hey! We are inside a function.'
     echo $1
```